

Design and implement a stream processing pipeline

Development phase

Markus Wohlgenannt

Sept 16, 2026

GitHub repository: https://github.com/Morwoi/Projekt_Data_Engineering_DLBDSEDE02

Implementation

The repository contains a containerized pipeline that streams simulated municipal sensor data into MongoDB via Kafka. Kafka runs in single node KRaft mode and MongoDB runs as an official container, both wired together with health checks in docker-compose.yml. A Python producer polls the Open-Meteo API for five simulated stations every 10 seconds and publishes each reading as JSON to the sensor-readings topic. When an API call fails, the producer switches to a local simulator that generates a plausible reading from the last known value and tags it as simulated, so the stream keeps running through a real outage instead of stalling. This fallback exists only because Open-Meteo stands in for sensor hardware that is not installed yet. It would not be appropriate with genuine sensors, where a real failure should be reported honestly as offline instead of being papered over with a synthesized value.

A Python consumer reads the topic and writes each reading into MongoDB as an idempotent upsert keyed on station and timestamp, committing Kafka offsets only after a successful write. Writes that fail are retried with backoff, and a message that exhausts all retries is parked in a dead letter collection instead of being dropped. Raw readings expire after a configurable retention period through a MongoDB TTL index.

Four scripts on top of the pipeline implement the actual query and reliability behaviour a planner and a citizen app would need. `planner_queries.py` rolls raw readings up into daily per station statistics computed from real readings only, with a coverage figure showing how much of a day rests on real measurements rather than fallback data. `citizen_status.py` classifies each station as ok, stale or unavailable based on the age of the last real reading rather than the last reading overall, since the fallback simulator alone would make a broken sensor look fine. `check_status.py` gives a quick host side health check, and `materializ_daily_stats.py` persists the daily aggregation past the raw data retention window. `verify_citizen_failover.py` forces a real outage against the running stack and confirms the status contract actually flips to unavailable and back, instead of assuming the logic is correct on paper.

Running `docker compose up --build` starts Kafka, MongoDB, the producer and the consumer, and the pipeline begins storing readings automatically with no manual setup steps needed. Full architecture and usage instructions are in the repository's README.

Feedback

The feedback on phase 1 was that the two user types were sketched too superficially and that Kafka's delivery guarantee alone does not say what a planner or a citizen actually needs from the data day to day. This was addressed with the query and reliability scripts above rather than more description, and the phase 1 user story was corrected to match.